

1 设计模式介绍

欢迎来到 设计模式

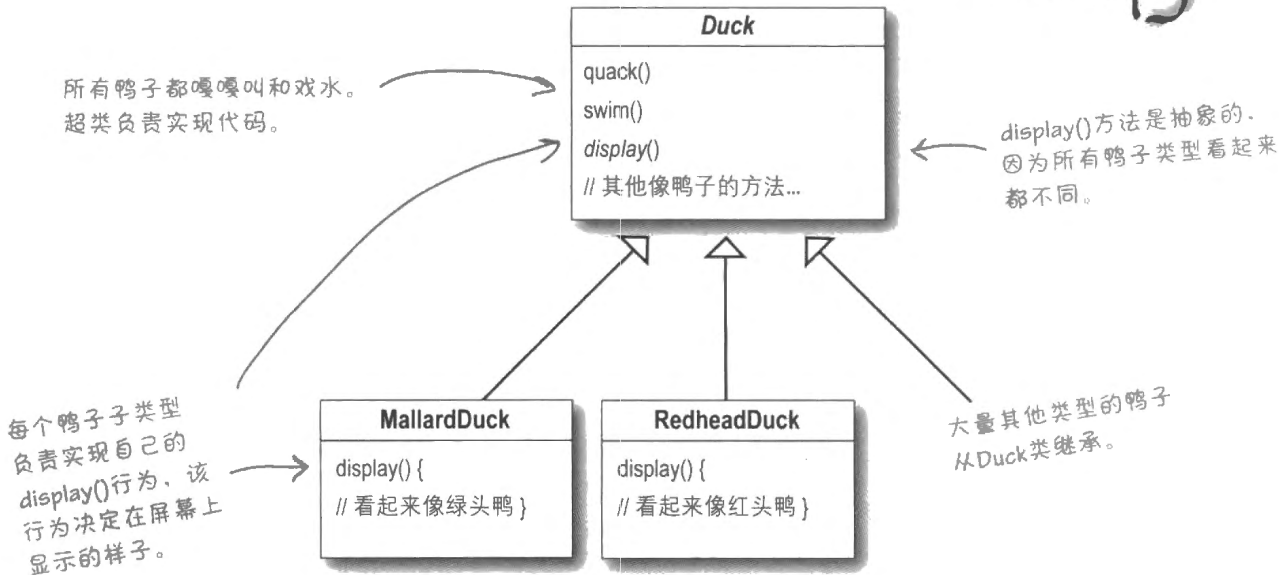
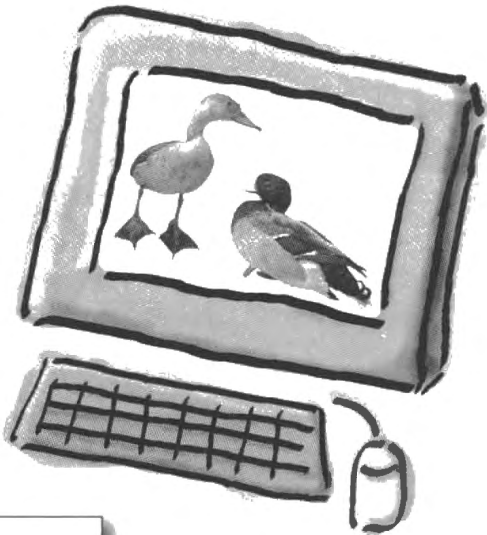


我们已经住在对象村，正要开始学习和使用设计模式……每个人都在这样做。很快我们就会成为Jim和Betty周三晚上模式小组的焦点了！

有人已经解决了你的问题。本章中，你将学到为何（以及如何）利用其他开发人员的智慧和教训。他们遭遇过同样的设计问题并且渡过了难关。本章结束前，我们会看看设计模式的用途和好处，以及一些关键的面向对象（OO）设计原则，并通过一个例子来了解模式是如何起作用的。使用模式的最佳方式是把它们装进大脑，在你的设计和已有的应用中识别用得上的地方。和代码复用不同，通过模式你获得了经验复用。

从简单的SimUDuck应用开始

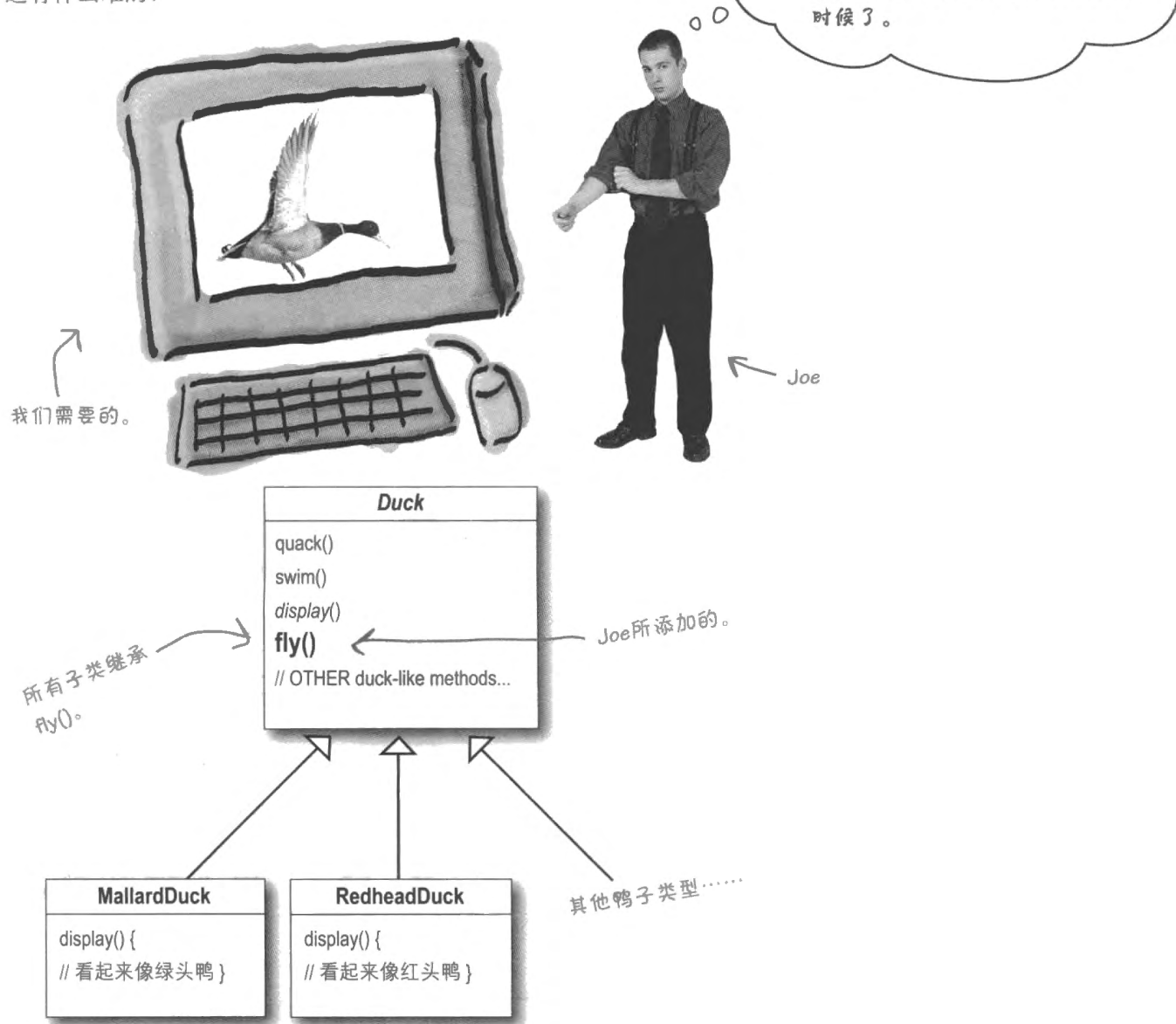
Joe上班的公司做了一款相当成功的鸭塘模拟游戏SimUDuck。游戏中会出现各种鸭子，一边戏水，一边嘎嘎叫。系统最初的设计师使用了标准的OO技巧，创建了一个Duck超类，然后所有其他鸭子类型继承它。



去年，公司面临的竞争压力越来越大。在为期一周的高尔夫假期兼头脑风暴会议后，公司管理层认为是时候做大的革新了。他们需要一些真正让人印象深刻的东西，在下周即将到来的毛伊岛股东会议上展示。

但现在我们需要鸭子飞

管理层决定，会飞的鸭子正是模拟器抛离竞争对手所需要的。当然，Joe的经理告诉管理层，Joe一个星期搞定没有问题。“毕竟，”Joe的老板说，“他是一个OO程序员……这有什么难的？”



但是，出大问题了……

Joe, 我在股东会议上。刚才的展示里有橡皮鸭子在屏幕上飞来飞去, 你是在开玩笑吗?



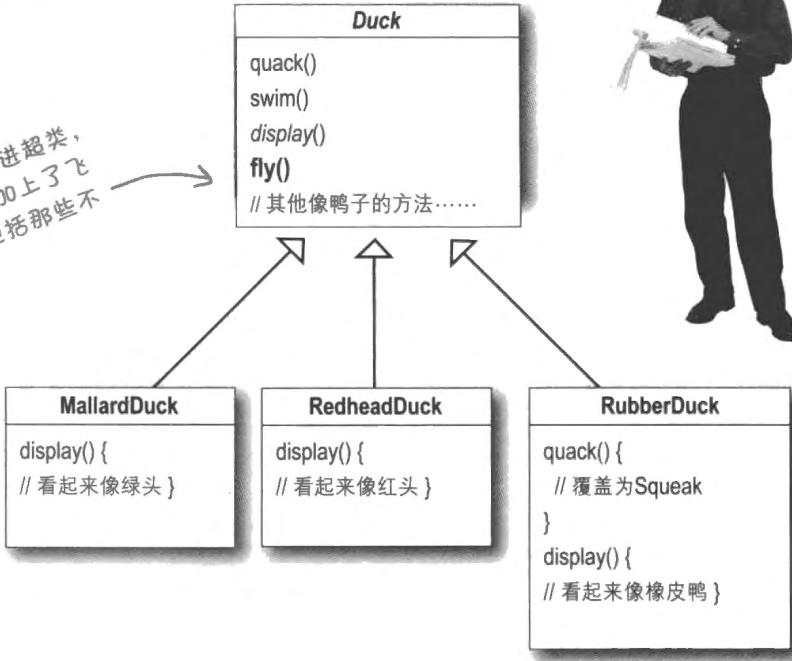
发生了什么事?

Joe没有注意到, 不是所有Duck的子类都应该会飞。当Joe给Duck超类添加新的行为时, 他也给某些Duck子类添加了不适合的行为。现在, SimUDuck程序中就有了会飞的无生命物体。

好吧, 我的设计里有小小毛病。不明白为什么他们不把它叫作“特色”, 还挺有趣的呀……

对代码的局部更新导致了非局部的副作用 (飞行的橡皮鸭) !

通过把fly()放进超类, 他为所有鸭加上了飞行能力, 包括那些不该飞的。



Joe想的是用继承来达到复用的目的, 当涉及到维护时, 效果并不那么好。

也要注意, 橡皮鸭不嘎嘎叫, 所以quack()被覆盖成“Squeak”。

Joe思考继承……

我可以总是覆盖橡皮鸭的fly()方法。就像对待quack()方法一样……



```

RubberDuck
quack() { // 吱吱叫 }
display() { // 橡皮鸭 }
fly() {
    // 覆盖为不做任何事
}
  
```

但当我们给程序添加木质诱饵鸭时，会发生什么？诱饵鸭不会飞也不会嘎嘎叫……



```

DecoyDuck
quack() {
    // 覆盖为不做任何事
}

display() { // 诱饵鸭 }

fly() {
    // 覆盖为不做任何事
}
  
```

这是继承层次中的另一个类。注意，和橡皮鸭相似，诱饵鸭不会飞，也不会嘎嘎叫。



削尖你的铅笔

以下哪些是使用继承来提供Duck行为的缺点？（多选）

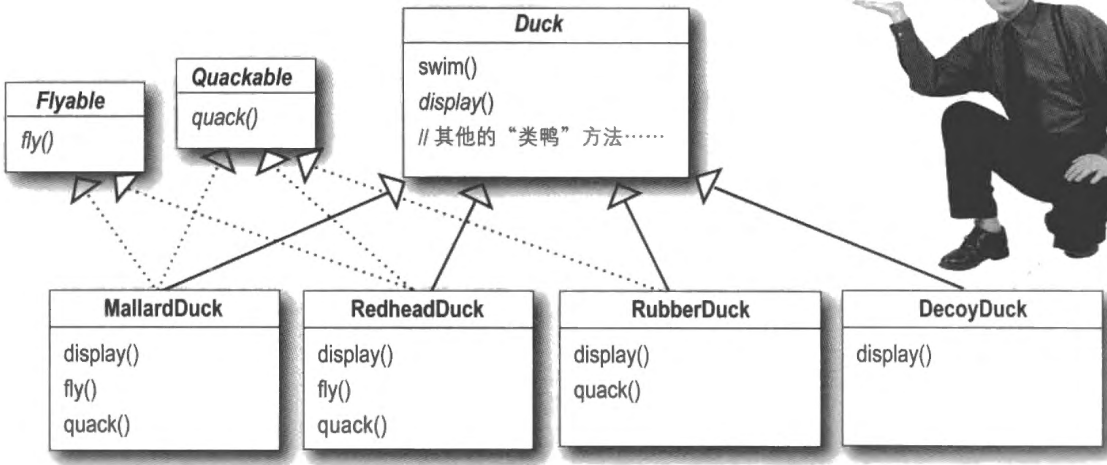
- ☐ A. 代码在多个子类重复。
- ☐ B. 运行时行为难以改变。
- ☐ C. 没法让鸭子跳舞。
- ☐ D. 难以获得所有鸭子行为的知识。
- ☐ E. 鸭子不能边飞边嘎嘎叫。
- ☐ F. 变更会不经意地影响其他鸭子。

接口如何?

Joe认识到继承可能不是答案，因为他刚刚收到一条备忘录，上面说管理层现在要每六个月更新产品（怎么个更新法他们还没定）。Joe知道需求规约会保持变化，每一次程序中添加了新的Duck子类，他都要被迫检查，有可能还要覆盖fly()和quack()，直到永远。

因此，他需要更干净的方式，只让某些（而不是全部）鸭子类型飞或嘎嘎叫。

我可以把fly()从Duck超类拿出来，并且做一个带fly()方法的Flyable()接口。这样，只有能飞的鸭子实现该接口，并且有一个fly()方法……同样，做一个Quackable，因为不是所有鸭子都能嘎嘎叫。



这个设计你觉得如何?

这好像是你想到的最蠢的主意了。干嘛不说“重复代码”算了？如果你认为不得不覆盖几个方法就算糟糕，当你需要对所有48个飞行Duck子类的飞行行为做小小改变时，你的感觉会怎样？



如果你是Joe，你会怎么做？

我们知道，不是所有子类都有飞行或嘎嘎叫行为，因此继承不是正确答案。让子类实现Flyable和/或Quackable解决了部分问题（没有了不恰当的飞行橡皮鸭），但它完全摧毁了这些行为的代码复用，因此，它只是创造了一个不同的维护噩梦。当然，还有可能出现：飞行鸭子有多种飞行行为……

这时候，你可能等待着设计模式骑着白马来拯救世界。但这样有什么意思呢？不，我们打算用老式的方法来勾勒出解决方案，通过应用良好的OO软件设计原则。

如果有一种构造软件的方法，当我们需要变更时，我们可以使用对现有代码影响最小的方式，那岂不是太美妙了？我们就可以花更少的时间重写代码，让程序去做更酷的事情……



软件开发中唯一的不变

OK，软件开发中，你唯一能深信不疑的是什么？

不管你在哪里工作，构造什么软件，用什么语言编程，一直伴随你、唯一不变的东西是什么？

变化

（用镜子来看答案）

不管你的应用设计得多好，随着时间推移，应用必定成长和变更，否则它就死了。



削尖你的铅笔

很多事情可以导致变更。列出一些你不得不修改应用代码的原因（我们写了一些我们自己的，给你起个头）。在往下走之前，用本章末尾的解答来检查答案。

顾客或用户决定需要别的东西，或者要新功能。

公司决定用另一家数据库厂商的产品，又从另一家供应商购买数据，数据格式不同，唉！

聚焦于问题……

我们知道，使用继承不能很好地解决问题，因为各个子类的鸭子行为一直在改变，让所有子类都拥有这些行为是不合适的。Flyable和Quackable接口一开始似乎还挺不错（只有真正会飞的鸭子才继承Flyable等），除了一点，Java接口通常没有实现的代码，因此没有代码复用。无论何时，何种情况下，如果你需要修改一个行为，常常被迫往下追踪到所有定义了该行为的子类并修改它，在这个过程中，可能会引入新的bug！

幸运的是，针对这种情况有一个设计原则。



设计原则

识别应用中变化的方面，把它们和不变的方面分开。



这是本书中众多设计原则中的第一个，本书后面我们还会花更多时间在这个原则上。

换句话说，如果每次有新的需求，某方面的代码就要变，那么你就知道了，这个行为需要抽出来，与其他不变的代码分离。

下面是这个原则的另一种思考方式：**把会变化的部分取出并封装，这样，以后你就可以修改或扩展这个部分，而不会影响其他不需要变化的部分。**

尽管这个概念很简单，它却是几乎每一个设计模式的基础。所有模式都提供了一种方式，让系统的某部分独立于其他部分而变化。

OK，是时候从Duck类抽出鸭子的行为了！

取出变化的部分，并把它“封装”起来，这样它就不会影响其他代码。

结果如何？代码变更引起的不经意后果变少，系统更加有弹性了！

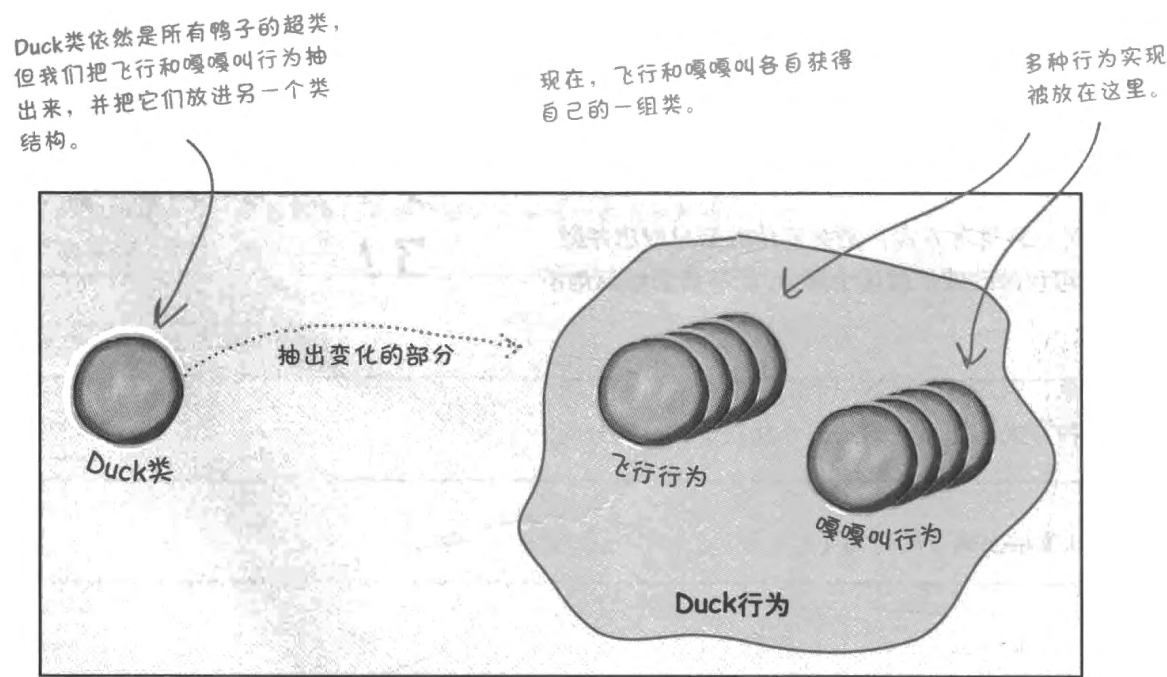
分离变和不变

从哪里开始呢？目前我们能辨别的是，除了fly()和quack()问题之外，Duck类还算正常，没有其他看起来经常变化的地方。因此，除了一些小的变更，我们打算先把Duck类放到一边。

现在，分离“变和不变的部分”。我们打算创建两组类（和Duck完全无关），一组和fly相关，另一组和quack相关。每一组类持有各自行为的所有实现。例如，我们可以有一个类实现嘎嘎叫，另一个类实现吱吱叫，另一个实现沉默不语。

我们知道fly()和quack()是Duck类中因不同鸭子而变化的部分。

为了从Duck类分离这些行为，我们把两个方法都从Duck类抽出，并创建一组新的类来表示每个行为。

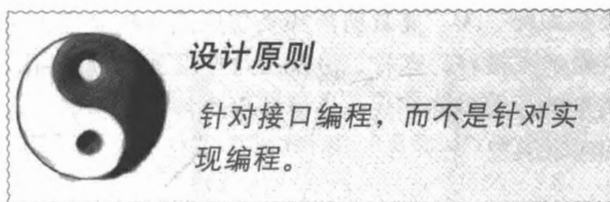


设计Duck的行为

那我们打算怎么设计那组实现飞行和嘎嘎叫行为的类？

我们希望保持各种东西的弹性，毕竟，一开始，是鸭子行为的僵化让我们陷入困境。我们也知道，我们要分配行为到Duck的实例。例如，我们可能要实例化一个新的MallardDuck实例，并在初始化时指定特定类型的飞行行为。既然我们这样做了，为什么不确保我们能够动态地改变鸭子的行为？换句话说，我们应该在Duck类中包含设置行为的方法，这样，我们就能够在运行时改变MallardDuck的飞行行为。

有了这些目标，我们来看看第二个设计原则：



我们将使用接口来表示每一个行为（例如，FlyBehavior和QuackBehavior），行为的每个实现将实现其中一个接口。

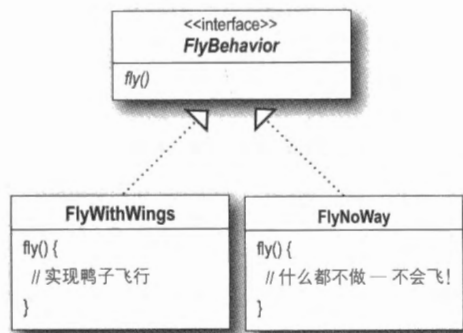
因此，这一次不是Duck类实现飞行和嘎嘎叫接口。我们专门做了一组类来表示行为（例如，“吱吱叫”），这就是行为类，由行为类而不是Duck类来实现行为接口。

这和我们之前所做的大有不同，行为既来自超类Duck的具体实现，也通过在子类自身中提供一个特化实现得到。两种情况都是依赖于实现。我们被锁定使用特定实现，没有改变行为的空间了（除非写更多的代码）。

在我们的新设计中，Duck子类将使用接口（FlyBehavior和QuackBehavior）所表示的行为，这样，行为实际的实现（换句话说，在类中编码的，实现FlyBehavior或QuackBehavior的特定具体行为）不会被锁定在Duck子类中。

从现在开始，Duck的行为将放在分离的类中，实现特定行为接口的类。

这样，Duck类不需要知道行为的任何实现细节。





我不明白为什么你非要把
FlyBehavior做成接口。抽象
超类也可以做同样的事情啊，
目的不就是使用多态吗？

“针对接口编程”真正的意思是“针对超类型编程。”

“接口”一词在这里有多个含义。接口是一个概念，也是Java的一个构造。针对接口编程不必真的使用Java接口。要点是通过针对超类型编程来利用多态，这样，实际的运行时对象不会被锁定到代码。我们可以重新描述“针对超类型编程”为“变量所声明类型应该是超类型，通常是抽象类或接口，这样，分配给这些变量的对象可以是超类型的任何具体实现，这意味着类声明时不必知道实际的对象类型!”

这可能对你来说不新鲜了，但只是为了确认我们在说同一件事情，下面是一个使用多态类型的简单例子，想象一个抽象类Animal，它有两个具体实现，Dog和Cat。

对实现编程是：

```
Dog d = new Dog();  
d.bark();
```

声明变量“d”为Dog类型（一个Animal的具体实现）强迫我们针对具体实现编程。

而针对接口/超类型编程则是：

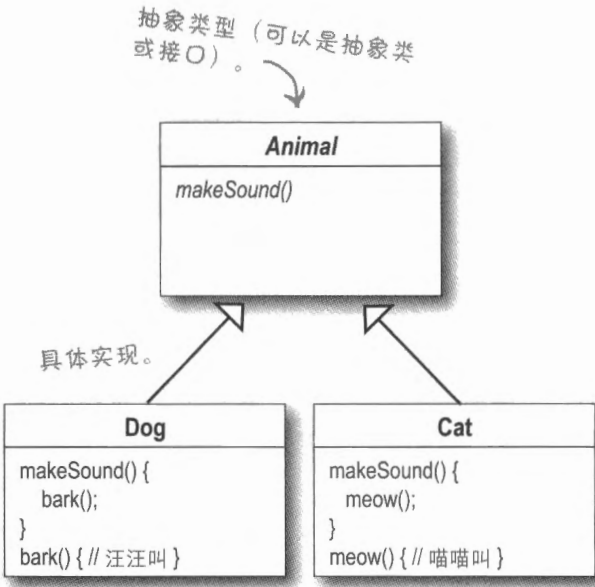
```
Animal animal = new Dog();  
animal.makeSound();
```

我们知道它是一个Dog，但我们可以多态地使用animal引用。

更棒的是，子类型实例化不用在代码中硬编码（像new Dog()），而是在运行时分配具体的实现对象：

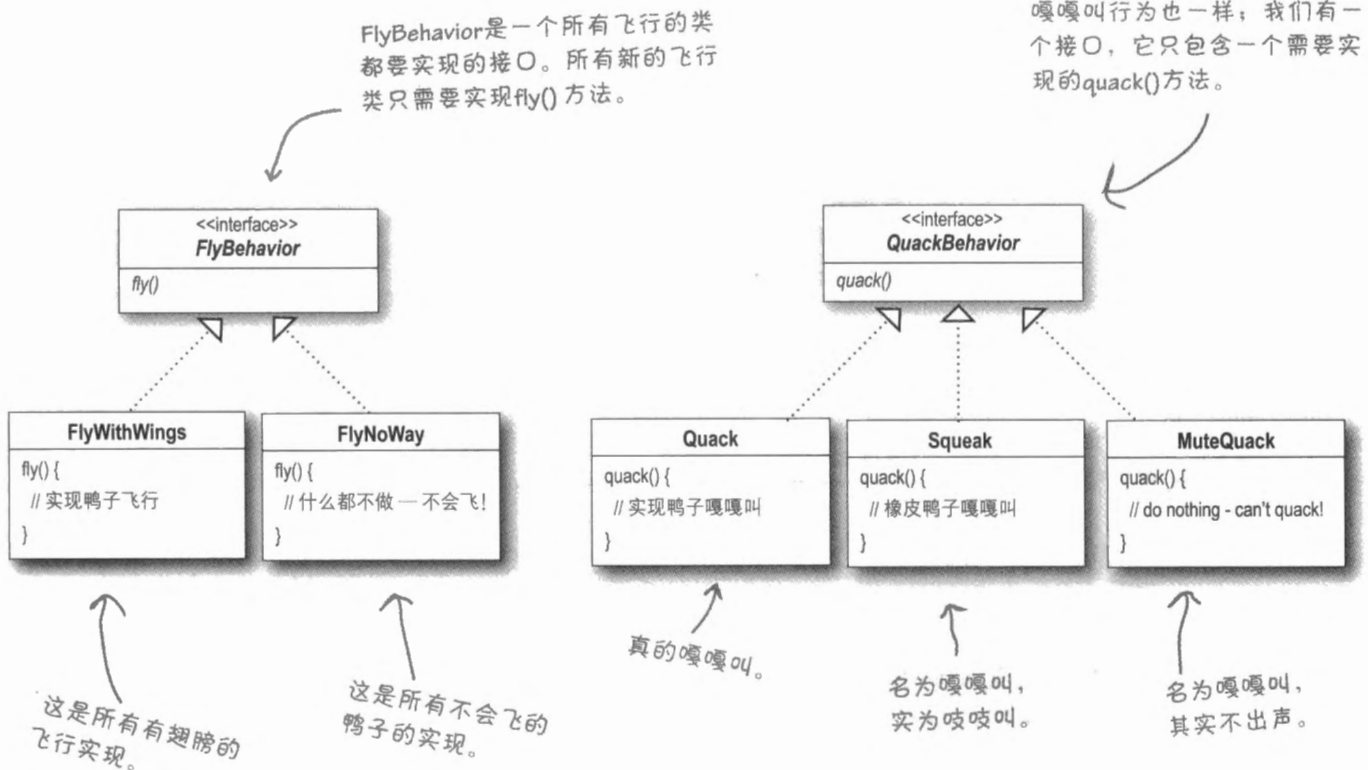
```
a = getAnimal();  
a.makeSound();
```

我们不知道实际的动物子类型，我们在意的只是它知道如何响应makeSound()。



实现Duck的行为

这里我们有两个接口，FlyBehavior和QuackBehavior，以及相应的实现每个具体行为的类：



通过这个设计，其他类型的对象可以复用飞行和嘎嘎叫行为，因为这些行为不再隐藏在Duck类后面了！

我们可以增加新的行为，不用修改任何已有行为类或涉及任何使用飞行行为的Duck类。

因此我们得到了复用的好处，却没有继承所带来的包袱。

没有蠢问题

问: 是否我总是要不得不先实现应用, 再看看什么地方有变化, 然后回去分离和封装这些地方?

答: 不总是这样, 通常你在设计应用时, 预测哪些区域会变化, 然后预先在代码中加入弹性。你会发现原则和模式可以应用在开发生命周期的任何阶段。

问: 应不应该把Duck也做成接口?

答: 本例中不要。正如你将看到的, 一旦我们把每样东西整合在一起, 不把Duck作为接口是有好处的, 让特定的鸭子, 像MallardDuck, 继承共同的属性和方法。

既然我们从Duck继承中移除了变化的部分, 就获得了继承结构的好处, 而不会带来问题。

问: 一个类只有行为, 感觉似乎有点怪异。类不是应该代表事物吗? 应该兼有状态“和”行为才对呀?

答: 在一个OO系统中, 是的, 类代表事物, 通常兼有状态(实例变量)和方法。在本例中, 事物碰巧是行为。但即使是行为, 也仍然可以有状态和方法。飞行行为可以有实例变量表达飞行行为的属性(翅膀每分钟拍几下、最大高度、速度等)。



削尖你的铅笔

- 1 使用我们的新设计, 如果你要给SimUDuck应用添加火箭动力飞行, 该怎么做?
- 2 你能想到一个使用Quack行为却不是鸭子的类吗?

1) 创建一个实现FlyBehavior接口的FlyRocketPowered类。
2) 一个例子是鸭鸣器(一个发出鸭叫声的装置)。

答案:

整合Duck的行为

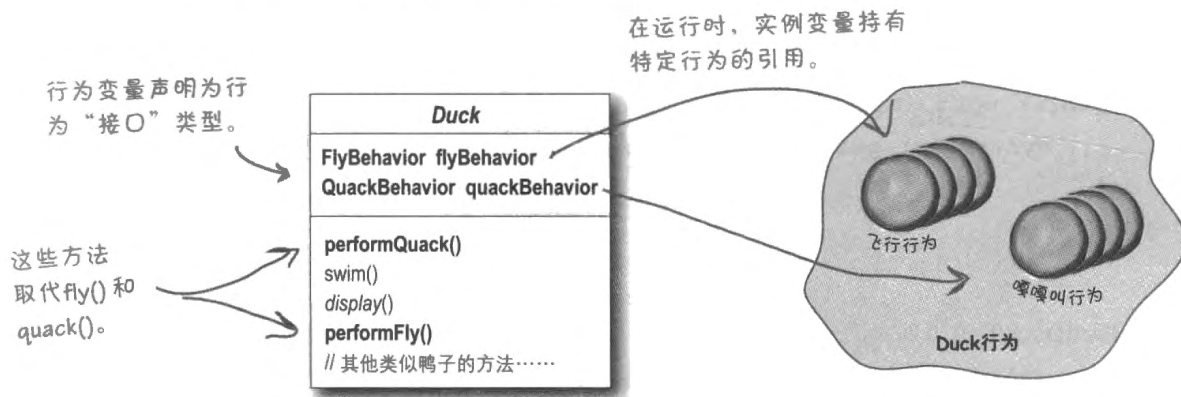
关键在于：**Duck**现在将委托其飞行和嘎嘎叫行为，而不是使用**Duck**类（或子类）中定义的嘎嘎叫和飞行方法。

做法如下：

- 首先，添加两个实例变量，类型为**FlyBehavior**和**QuackBehavior**，名称为**flyBehavior**和**quackBehavior**。在运行时，每个具体鸭子对象将给这些变量分配特定行为，像飞行的**FlyWithWings**，嘎嘎叫的**Squeak**。

我们也将从**Duck**类（以及任何子类）移除**fly()**和**quack()**方法，因为我们已经把这些行为搬进**FlyBehavior**和**QuackBehavior**类。

我们将用两个类似的方法**performFly()**和**performQuack()**来替换**Duck**类中的**fly()**和**quack()**，接下来就会看到它们是怎么工作的。



- 现在，我们实现**performQuack()**:

```
public abstract class Duck {
    QuackBehavior quackBehavior;
    // 更多

    public void performQuack() {
        quackBehavior.quack();
    }
}
```

每个Duck引用实现QuackBehavior接口的某些东西。

Duck对象不自己处理嘎嘎叫行为而是把行为委托给quackBehavior引用的对象。

相当简单，嗯哼？为了执行嘎嘎叫，Duck只要让quackBehavior所引用的对象为它嘎嘎叫即可。在这部分代码中我们不关心具体Duck是哪种对象，只要它知道怎么quack()就行了！

更多整合.....

- 3 OK, 是时候操心如何设置flyBehavior和IquackBehavior实例变量了。我们看看MallardDuck类:

```
public class MallardDuck extends Duck {
```

```
    public MallardDuck() {  
        quackBehavior = new Quack();  
        flyBehavior = new FlyWithWings();  
    }
```

记住, MallardDuck继承来自类Duck的quackBehavior和flyBehavior实例变量。

MallardDuck使用Quack类来处理嘎嘎叫, 因此, performQuack()被调用时, 嘎嘎叫的责任被委托给Quack对象, 我们得到了真正的嘎嘎叫。

使用FlyWithWings作为其FlyBehavior类型。

```
    public void display() {  
        System.out.println("I'm a real Mallard duck");  
    }  
}
```

MallardDuck的嘎嘎叫是真正活生生的嘎嘎叫, 不是吱吱叫, 也不是叫不出声。当MallardDuck被实例化时, 其构造器初始化MallardDuck继承来的quackBehavior实例变量为一个新的Quack (QuackBehavior的一个具体实现类) 类型实例。

鸭子的飞行行为也一样, MallardDuck的构造器初始化继承来的flyBehavior实例变量为FlyWithWings (FlyBehavior的一个具体实现类) 类型的实例。



等一下，你不是说
不应该针对实现编程吗？但我们现在在
构造器里面做什么？我们弄了一个具体
Quack实现类的新实例！

被抓到了，我们的确是这么做的……

目前为止。

在本书后续内容中，我们的工具箱里会有更多的模式，可以帮助我们修正这一点。

还是要注意，虽然我们把行为设置为具体的类（通过实例化像Quack或 FlyWithWings这样的行为类，并把它赋值给行为引用变量），我们还是可以轻易在运行时做出改变。

因此，我们依然有很多弹性，只不过我们在初始化实例变量上做得很糟糕。但想一想，既然 quackBehavior 实例变量是一个接口类型，运行时，我们可以（通过多态的魔力）动态赋值不同的QuackBehavior实例类。

花一点时间考虑如何实现一只鸭子，其行为可以在运行时改变（几页以后，你会看到做这件事的代码）。

测试Duck的代码

- 1 输入并编译以下Duck类 (Duck.java)，以及两页前的MallardDuck类 (MallardDuck.java)。

```
public abstract class Duck {  
  
    FlyBehavior flyBehavior;  
    QuackBehavior quackBehavior;  
    public Duck() { }  
  
    public abstract void display();  
  
    public void performFly() {  
        flyBehavior.fly();  
    }  
  
    public void performQuack() {  
        quackBehavior.quack();  
    }  
  
    public void swim() {  
        System.out.println("All ducks float, even decoys!");  
    }  
}
```

为行为接口类型声明两个引用变量。所有鸭子子类（在一个包中）都继承它们。

委托给行为类。

- 2 输入和编译FlyBehavior接口 (FlyBehavior.java) 以及两个行为实现类 (FlyWithWings.java和FlyNoWay.java)。

```
public interface FlyBehavior {  
    public void fly();  
}
```

所有飞行行为类实现的接口。

```
public class FlyWithWings implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying!!");  
    }  
}
```

确实能飞的鸭子的飞行行为实现……

```
public class FlyNoWay implements FlyBehavior  
{  
    public void fly() {  
        System.out.println("I can't fly");  
    }  
}
```

不会飞的鸭子的飞行行为实现（像橡皮鸭和诱饵鸭）。

测试Duck的代码，继续……

- ③ 输入和编译QuackBehavior接口（QuackBehavior.java）和三个行为实现类（Quack.java、MuteQuack.java和Squeak.java）。

```
public interface QuackBehavior {
    public void quack();
}

public class Quack implements QuackBehavior {
    public void quack() {
        System.out.println("Quack");
    }
}

public class MuteQuack implements QuackBehavior {
    public void quack() {
        System.out.println("<< Silence >>");
    }
}

public class Squeak implements QuackBehavior {
    public void quack() {
        System.out.println("Squeak");
    }
}
```

- ④ 输入和编译测试类（MiniDuckSimulator.java）。

```
public class MiniDuckSimulator {
    public static void main(String[] args) {
        Duck mallard = new MallardDuck();
        mallard.performQuack();
        mallard.performFly();
    }
}
```

这里调用MallardDuck继承来的performQuack()方法，然后委托给该对象的QuackBehavior[也就是说，调用继承来的quackBehavior引用的quack()]。

MallardDuck继承来的performFly()方法也是一样。

- ⑤ 运行代码！

```
File Edit Window Help Yadayadayada
%java MiniDuckSimulator
Quack
I'm flying!!
```

动态设置行为

在鸭子中建立了这些动态的能力，却没有在使用，实在可惜!想象一下，如果你要通过一个Duck类的setter方法设置鸭子的行为类型，而不是在鸭子的构造器中实例化。

1 添加两个新的方法到Duck类:

```
public void setFlyBehavior(FlyBehavior fb) {  
    flyBehavior = fb;  
}  
  
public void setQuackBehavior(QuackBehavior qb) {  
    quackBehavior = qb;  
}
```

Duck
FlyBehavior flyBehavior QuackBehavior quackBehavior
swim() display() performQuack() performFly() setFlyBehavior() setQuackBehavior() // 其他像鸭子的方法……

任何时候我们要改变鸭子的飞行行为，我们可以调用这些方法。

编辑注：可能会有歧义——改掉。

2 做一个新的Duck类型 (ModelDuck.java)。

```
public class ModelDuck extends Duck {  
    public ModelDuck() {  
        flyBehavior = new FlyNoWay();  
        quackBehavior = new Quack();  
    }  
  
    public void display() {  
        System.out.println("I'm a model duck");  
    }  
}
```

一开始，我们的模型鸭不会飞……

3 做一个新的FlyBehavior类型 (FlyRocketPowered.java)。

```
public class FlyRocketPowered implements FlyBehavior {  
    public void fly() {  
        System.out.println("I'm flying with a rocket!");  
    }  
}
```

没关系，我们创建一个火箭动力的飞行行为。



④ 改变测试类 (MiniDuckSimulator.java)，添加ModelDuck，使ModelDuck具有火箭动力。

```
public class MiniDuckSimulator {
    public static void main(String[] args) {
        Duck mallard = new MallardDuck();
        mallard.performQuack();
        mallard.performFly();
```

```
        Duck model = new ModelDuck();
        model.performFly();
        model.setFlyBehavior(new FlyRocketPowered());
        model.performFly();
    }
}
```

如果行得通，模型鸭就能动态改变其飞行行为，如果是在Duck类中实现，就做不到这样了。

⑤ 运行它！

```
File Edit Window Help Yabbadabbadoo
%java MiniDuckSimulator
Quack
I'm flying!!
I can't fly
I'm flying with a rocket!
```

改变前



第一次调用performFly()会被委托给flyBehavior对象。该对象是一个FlyNoWay实例，在ModelDuck构造器中设置。

此处调用model继承来的行为setter方法，然后……没错！model突然就有了火箭动力飞行能力！



改变后

要在运行时改变鸭子的行为，只需调用鸭子的setter方法。

封装行为的全貌

OK，现在我们已经深入研究了鸭子模拟器的设计，是时候浮出水面看一看全貌了。

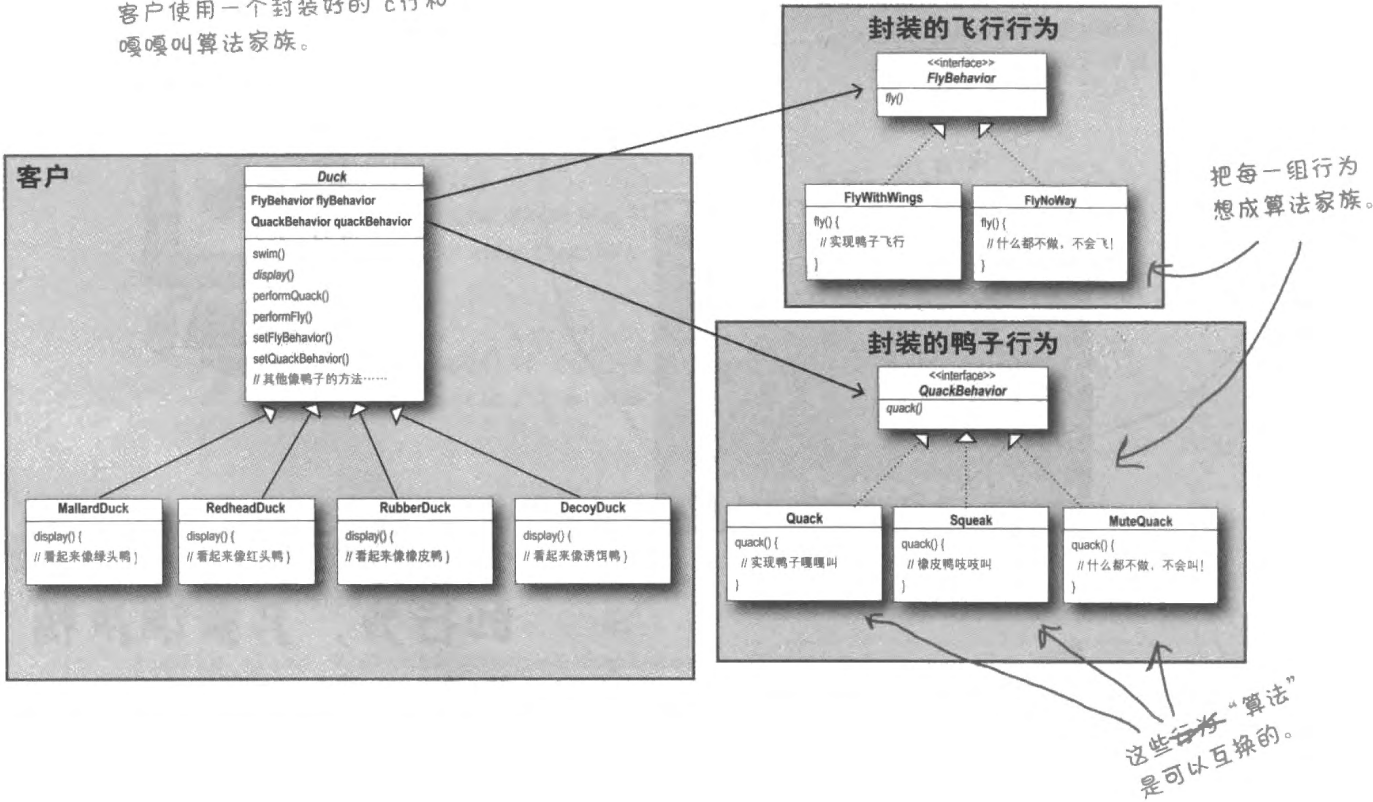
以下是重新设计后的整个类结构，里面有你所期望的一切：鸭子扩展Duck，飞行行为实现FlyBehavior，嘎嘎叫行为实现QuackBehavior。

也要注意，我们描述事情的方式有了一点不同。不再把鸭子的行为看作一组行为，而是开始把它们看作一个算法家族。想想看：在SimUDuck设计中，算法代表鸭子会做的事（不同的嘎嘎叫或飞行方式），不过我们也可以轻易地把同样的技巧用于一组实现计算不同州销售税的类。

特别要注意类之间的关系。抓起你的笔，在类图的每个箭头上写出合适的关系[IS-A（是一个），HAS-A（有一个），和IMPLEMENTS（实现）]。

确认你做这件事。

客户使用一个封装好的飞行和嘎嘎叫算法家族。



HAS-A比IS-A好

HAS-A关系很有趣：每只鸭子有一个FlyBehavior和一个QuackBehavior，以便委托飞行和嘎嘎叫。

当你把两个类像这样放在一起时，你在使用组合。不继承行为，鸭子通过组合正确的行为对象获得它们的行为。

这是一个重要的技巧；事实上，它是我们的第三个设计原则的基础：



设计原则

优先使用组合而不是继承。

正如你已经看到的，使用组合创建系统给了你更大的弹性。不仅是让你把一个算法家族封装进它们自己的一组类，而且让你在运行时改变行为，只要组合的对象实现正确的行为接口。

组合用在许多设计模式中，在本书前前后后，你会看到更多关于其优点和缺点的内容。



鸭鸣器是一种猎人用来模拟鸭子嘎嘎叫的装置。你如何实现自己的鸭鸣器，而不用从Duck类继承？



大师和徒弟……

大师：告诉我，在面向对象的路上，你学到了什么？

徒弟：大师，我学到了，面向对象的承诺是复用。

大师：继续…

徒弟：大师，通过继承，可以复用所有好东西，所以我们可以大量缩减开发时间，就像在林中砍竹子一样迅速。

大师：开发完成之前还是之后，花费在代码上的时间更多呢？

徒弟：答案是之后，大师。比起初始的开发，我们总是花费更多时间维护和改变软件。

大师：因此，精力是不是应该花费在维护和可扩展性的复用上？

徒弟：大师，我相信这就是真相。

大师：我觉得你还有很多东西要学。希望你再进一步对继承做冥想。正如你所见到的，继承有它的问题，还有其他方式可以达到复用。

说起设计模式……



恭喜你学会了
第一个模式！

你刚刚应用了第一个设计模式——策略模式。对的，你使用策略模式改写了SimUDuck应用。

多亏有了这个模式，模拟器可以应对任何变化，管理层可以计划他们的毛伊岛之旅了。

为了学会这个模式，我们让你走了很长一段路，下面是这个模式的正式定义：

策略模式定义了一个算法族，分别封装起来，使得它们之间可以互相变换。策略让算法的变化独立于使用它的客户。

如果你想给朋友留下深刻印象和影响管理层关键人物，就使用这个定义。



设计谜题

以下是一堆杂乱的类和接口，来自一个动作冒险游戏。你会发现，里面有代表游戏角色的类以及角色可以使用的武器行为的类。每个角色一次只能使用一件武器，但是可以在游戏过程中任何时间换武器。你的工作是把它们整理好……

(答案在本章末尾。)

你的任务：

- ❶ 排列这些类。
- ❷ 找出一个抽象类、一个接口和八个类。
- ❸ 在类之间画箭头。
 - a. 继承（“扩展”）画成 
 - b. 接口（“实现”）画成 
 - c. HAS-A画成 
- ❹ 把方法 `setWeapon()` 放进正确的类。



在当地餐厅偷听到……

Alice

我要一份涂了奶酪以及果酱的白面包、加了香草冰淇淋的巧克力苏打、夹了培根的火烤芝士三明治、吞拿鱼沙拉吐司、香蕉船（有冰淇淋和香蕉片）、一杯加了奶油和两颗糖的咖啡……嗯……还有一个烧烤汉堡！

Flo

给我一份C.J. White，一个黑与白，一份Jack Benny，一杯Radio，一份主厨船，一杯普通咖啡，还有给我烧一个！



这两人点的餐有什么不同？没什么不同！点的东西都一样，只是Alice用词多了一倍，就要挑战脾气暴躁的快餐店厨师的底线了。

什么是Flo有而Alice没有的？和快餐店厨师之间的共享词汇。共享词汇不仅使得和厨师沟通更容易，也让厨师不用记这么多东西，因为这些餐点模式已经在他的大脑里了。

设计模式让你和其他开发人员之间有共享的词汇。一旦懂了这些词汇，你可以更容易地和其他开发人员沟通，并激发不懂设计模式的开发人员开始学习设计模式。通过在模式层面而不是琐碎的对象层面上思考，设计模式也提升了你关于架构的思考层次。

在办公室隔间偷听到……

因此我创建了这个广播类。它追踪所有监听它的对象，任何时候只要有新数据进来，它就发送消息给每位监听者。酷的地方是，监听者可以随时加入此广播系统，甚至可以随时将自己移除。这真的是非常动态和松耦合！

Rick



BRAIN POWER

除了OO设计和餐间谈话之外，你还能够想到哪些共享词汇？（暗示：想一想汽修、木工、料理、航管）利用这些行话进行沟通的质量如何？

你能否想到OO设计的某方面是用模式名称沟通的？用“策略模式”这个名字沟通的质量如何？

Rick，为什么你不直接说你在使用“观察者模式”？



没错，如果你用模式来沟通，其他开发人员马上准确地知道你在描述的设计。只不过，不要得了“模式热”……如果连写一个“Hello World”都开始使用模式，那就代表你已经得了这病了。

共享模式词汇的威力

使用模式和沟通的意义不只是共享“行话”。

共享模式词汇“威力强大”。当你使用模式和另一个开发人员或者你的团队沟通时，你沟通的不只是模式名称，还有一整套模式所代表的质量、特性和约束。

模式能让你用更少的话表达更多的意思。当你在描述中使用模式的时候，其他开发人员会很快地准确了解你大脑中的设计。

在模式层次上说话，能让你停留在“设计思维”的时间更长一些。使用模式谈论软件系统，能让你把讨论保持在设计层次，不必下潜到实现对象和类的琐碎细节。

共享词汇可以给你的开发团队带来“涡轮增压”的效果。对于设计模式有深入了解的团队，相互之间产生误解的可能性更少，能更快速地向前走。

共享词汇激励更多初级开发人员快速成长。初级开发人员向有经验的开发人员看齐。当高级开发人员使用设计模式时，初级开发人员也会被推动来学习设计模式，从而在你的组织建立一个模式用户的社群。

“我们使用策略模式实现鸭子的各种行为。”这句话告诉你，鸭子的行为已经被封装进入一组自己的类中。这些类可以被轻易扩展和改变，如果有需要的话，甚至在运行时也可以。

有多少次你参加过的设计会议，话题很快降级到实现细节中？

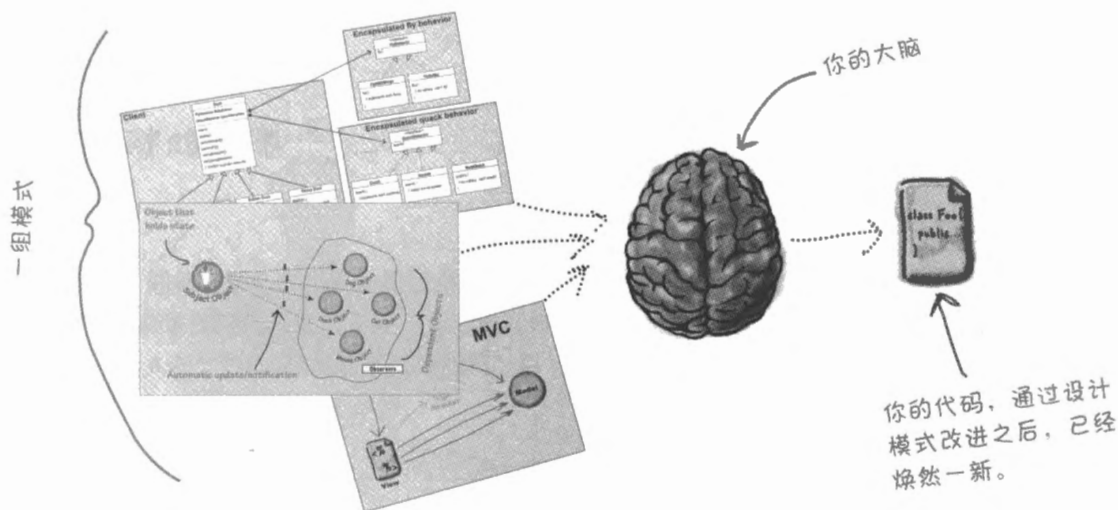
当你的团队开始通过模式术语分享设计思路和经验时，你就建起了一个模式用户的社群。

考虑在你的组织内发起一个模式学习小组，说不定在学习的过程中，就得到回报了……

我如何使用设计模式?

我们都使用现成的库和框架。我们针对它们的API写一些代码，编译进我们的程序，享受别人写好的一大堆代码带来的好处。想一想Java API以及它们带来的功能：网络、GUI、IO等。库和框架在开发模型中扮演很重要的角色，我们可以挑选组件，把它们插进正确的地方。但是……库和框架不能帮助我们把自己的应用组织成更容易理解、更容易维护、更有弹性的结构。这是设计模式起作用的地方。

设计模式不直接进入你的代码，它们首先进入你的“大脑”。一旦你在大脑中装入管用的模式知识，你就能够开始在新设计中应用它们。当你发现你的旧代码堕落成没有弹性的烂摊子，可以用设计模式改写。



没有蠢问题

问：如果设计模式这么棒，为什么没有人创建一个设计模式库，这样我们就不必自己动手了？

答：设计模式比库的等级更高。设计模式告诉我们如何组织类和对象来解决某些问题，采纳这些设计并让它们匹配我们的特定应用，是我们的工作。

问：库和框架不也是设计模式吗？

答：框架和库不是设计模式；它们提供和我们代码相关的特定实现。但是，有时库和框架会在其实现中使用设计模式。这很棒，因为一旦你理解了设计模式，你会更快理解，这些API是围绕着设计模式构造的。

问：因此，没有设计模式的库？

答：没有，但是稍后你会学习带有模式列表的模式目录，你可以在你的应用中使用它们。



将信将疑的开发人员



友善的模式大师

开发人员：OK，但是不都只是良好的面向对象设计吗？我的意思是，只要我遵循封装原则，而且懂得抽象、继承和多态，我还真的需要考虑设计模式吗？这不是相当直白的事情吗？我过去上的这些OO课程是干什么的？我认为设计模式只对那些不懂良好OO设计的人有用。

大师：啊，这是针对面向对象开发产生的真正误解中的一个：以为知道OO基础概念，就会自动精通如何建造有弹性、可复用和可维护的系统。

开发人员：不是这样吗？

大师：不是。事实证明，构造这样的OO系统的方法不总是那么容易发现的，需要艰苦工作来挖掘。

开发人员：我想我开始懂了。这些有时候比较隐晦的构造面向对象系统的方式被收集起来……

大师：……是的，就成了一组“设计模式”。

开发人员：因此，懂得了模式，我就可以省去艰苦的劳动，直扑那些总是行得通的设计？

大师：一定程度上是这样的，不过要记得，设计是一种艺术，总是有要折中的地方。但是，如果你遵循经过深思熟虑而且经历时间考验的设计模式，你就领先别人了。

开发人员：如果我找不到模式，那怎么办呢？

记住，知道类似抽象、继承和多态这些概念，并不会让你成为好的面向对象设计人员。设计大师考虑的是创建弹性的设计，可以维护，可以应付改变。



大师：模式下面藏有一些面向对象原则。当你找不到你的问题所匹配的模式时，懂得这些原则会帮助你渡过难关。

开发人员：原则？你是说除了抽象、封装...之外，还有...

大师：是的，建立可维护的OO系统的要诀之一，就是考虑它们将来可能会怎样变化，这些原则处理这样的问题。



你的设计工具箱中的工具

你几乎快要读完第一章了！你已经在你的OO工具箱中放入了一些工具，在我们转到第2章之前，先将这些工具一一列出。

OO基础

抽象
封装
多态
继承

我们假设你知道OO基础，像抽象、封装、多态和继承。如果你觉得有点生疏了，拿出你最喜欢看的面向对象书籍复习，然后再略读本章一次。

OO原则

封装变化。

优先使用组合而不是继承。

针对接口编程，而不是针对实现编程。

在后续的内容，我们会更详细地看这些原则，也会添加一些原则到清单上。

OO模式

策略-定义算法家族，分别封装起来，让它们之间可以互相替换。策略模式让算法的变化独立于使用算法的客户。

搞定一个，还有更多！

要点

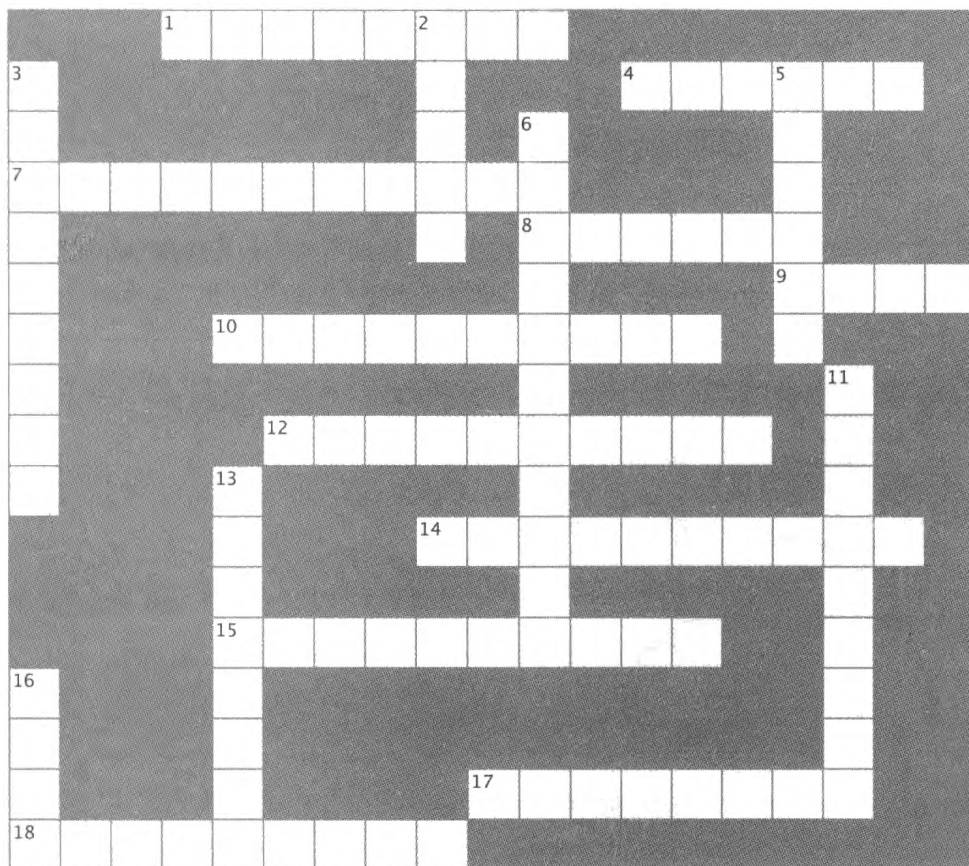
- 知道OO基础，还不能让你成为好的OO设计人员。
- 良好的OO设计是可复用、可扩展和可维护的。
- 模式向你展示如何建造具有良好OO设计质量的系统。
- 模式是历经验证的OO经验。
- 模式不给你代码，而是针对设计问题给出通用的解决方案。你把它们用到特定的应用中。
- 模式不是被发明，而是被发现。
- 大多数的模式和原则，都着眼于软件中的变化这个主题。
- 大多数模式允许系统的某些部分独立于其他部分而变化。
- 我们经常尝试把系统中会变化的部分抽出来封装。
- 模式提供了一种共享的语言，能最大化你和其他开发人员沟通的价值。



设计模式填字游戏

给你的右脑找些事情做。

标准的填字游戏，答案所用的词都来自本章。



横排

1. 模式可以帮助我们建造_____应用。
4. 策略可以是_____。
7. _____优于继承。
8. 开发的常态。
9. Java IO，网络，声音。
10. 大多数模式遵从OO_____。
12. 设计模式是被分享的_____。
14. 高层库。
15. 从其他人的_____学习。
17. 修正模拟器的模式。
18. 对_____编程，而不是对实现编程。

竖排

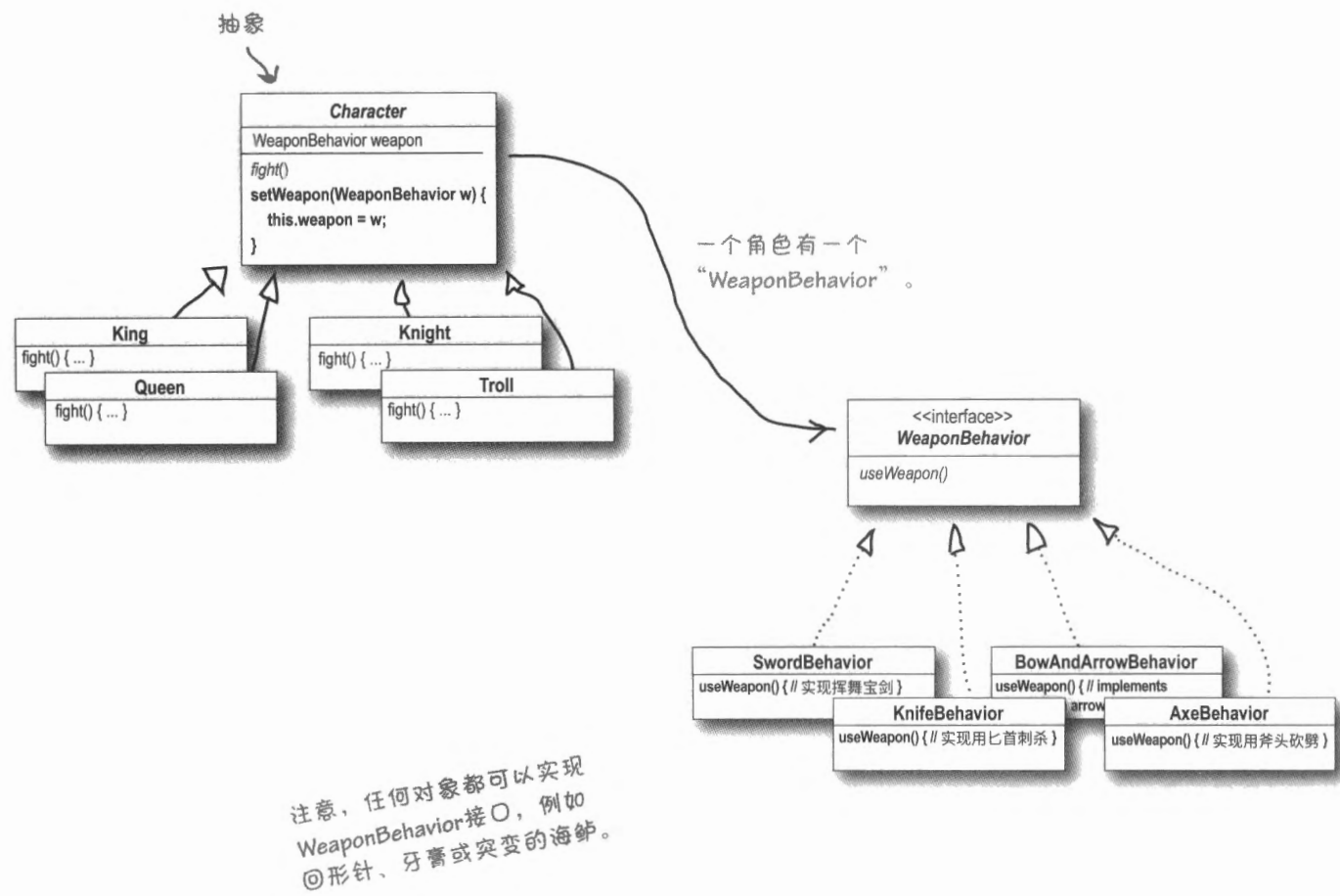
2. 模式进入你的_____。
3. 不会呱呱叫的鸭子。
5. 橡皮鸭的叫声_____。
6. _____变化的东西。
11. 烤乳酪带培根。
13. Rick为这个模式兴奋不已。
16. Duck的演示发生在这个地方。



设计谜题解答

Character（角色）是所有其他角色的抽象类，这些角色包括国王（King）、皇后（Queen）、骑士（Knight）和妖怪（Troll）。而 Weapon（武器）行为是一个接口，所有武器行为实现它。因此，所有实际的角色和武器都是具体类。

为了换武器，每个角色调用setWeapon()方法，此方法在Character超类中定义。在打斗（fight）过程中，会调用给定角色所持当前武器的useWeapon()方法，给另一个角色带来巨大的肉体伤害。





削尖你的铅笔

答案

以下哪些是使用子类化来提供特定的Duck行为的缺点？（多选）解答如下。

- ☒ A. 代码在多个子类重复。
- ☒ B. 运行时行为难以改变。
- ☐ C. 没法让鸭子跳舞。
- ☒ D. 难以获得所有鸭子行为的知识。
- ☐ E. 鸭子不能边飞边嘎嘎叫。
- ☒ F. 变更会不经意地影响其他鸭子。



削尖你的铅笔

答案

你的应用中有哪些因素会驱动变更？你可能有一个差别很大的清单，以下是我们的一些因素，看起来相似吗？解答如下：

顾客或用户决定需要别的東西，或者要新功能。

公司决定用另一家数据库厂商的产品，还从另一家供应商购买数据，数据格式不同，唉！

好吧，技术变了，我们要更新代码才能使用新协议。

我们学到了足够的构造系统的知识，希望回头去把事情做得更好一些。



设计模式填字游戏解答

